

20. LangChain & Agentic AI – From Basics to Advanced 20

Class 20 Running Notes 16-09-2026

1. What Is a Chain?

A Chain is a sequence of connected components where the output of one component becomes input to another component.

Example:

Prompt



Model



Parser

A Chain connects multiple processing steps into one workflow.

2. What Is Composition?

Composition means connecting smaller components together to build a bigger workflow.

Example:

We already have:

Prompt

Model

Parser

These are individual components.

Composition means:

Prompt + Model + Parser



One Chain

Another example from real life:

Engine

Wheels

Steering

Brakes



Car

Individual components combine to create a larger system.

Similarly:

Prompt

Model

Parser



Chain

3. What Is LCEL?

LCEL stands for: LangChain Expression Language

LCEL is LangChain's declarative way to compose Runnable components into chains.

ie LCEL helps us connect LangChain components into pipelines.

Example:

```
chain = prompt | model | parser
```

That's LCEL composition.

4. First Understand the Manual Approach

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    "Explain {topic} in very simple English."
```

```
)
```

```
model = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
parser = StrOutputParser()
```

```
prompt_value = prompt.invoke(
```

```
    {"topic": "Python"}
```

```
)
```

```
response = model.invoke(prompt_value)
```

```
result = parser.invoke(response)
```

```
print(result)
```

5. First LCEL Program

Now let us convert our manual program into LCEL.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    "Explain {topic} in very simple English."
```

```
)
```

```
model = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
result = chain.invoke(
```

```
    {"topic": "Python"}
```

```
)
```

```
print(result)
```

6. Compare Both Chains

6.1 Two-Component Chain

chain = prompt | model

Flow:

Prompt

↓

Model

↓

AIMessage

6.2 Three-Component Chain

chain = prompt | model | parser

Flow:

Prompt

↓

Model

↓

Parser

↓

String

compare types also and explain clearly

7. Manual Approach vs LCEL

Manual Approach	LCEL
Call each step yourself	Connect steps into pipeline
Handle intermediate values yourself	Chain passes outputs automatically
More explicit code	More compositional code
Fine for simple learning/debugging	Useful for reusable LangChain workflows

Very important:

Manual approach is not wrong.

LCEL gives another, often cleaner, way to compose LangChain components.

8. Interactive LCEL Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} in very simple English.
```

```
    Give:
```

```
    - Simple definition
```

```
    - 3 important points
```

```
    - One example
```

```
    """
```

```
)
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
topic = input("Enter topic: ")
```

```
result = chain.invoke(  
    {"topic": topic}  
)
```

```
print("\nAI Response:")
```

```
print(result)
```

9. Complete Multiple-Variable Program

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = PromptTemplate.from_template(
    """
    Explain {topic} in {language}
    for a {level} learner.

    Give one simple example.
    """
)

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

parser = StrOutputParser()

chain = prompt | model | parser

topic = input("Enter topic: ")
language = input("Enter language: ")
```

```
level = input("Enter learner level: ")
```

```
result = chain.invoke(  
    {  
        "topic": topic,  
        "language": language,  
        "level": level  
    }  
)
```

```
print("\nFinal Answer:")
```

```
print(result)
```

10. Practical Application 1 — Interview Helper

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(  
    """
```

```
    Generate {count} {level} interview questions  
    about {technology}.
```

```
    Use simple English.
```

```
    """)
```

```

)

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

parser = StrOutputParser()

chain = prompt | model | parser

technology = input("Enter technology: ")
level = input("Enter level: ")
count = input("How many questions: ")

result = chain.invoke(
    {
        "technology": technology,
        "level": level,
        "count": count
    }
)

print("\nInterview Questions:")
print(result)

```

11. Practical Application 2 — Travel Assistant

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Create a {days}-day travel plan for {place}.
```

```
    Budget: {budget}
```

```
    Main Interest: {interest}
```

```
    Keep the plan simple.
```

```
    """
```

```
)
```

```
model = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```

result = chain.invoke(
    {
        "days": 3,
        "place": "Goa",
        "budget": "Medium",
        "interest": "Beaches"
    }
)

print(result)

```

12. Practical Application 3 — Recipe Generator

```

import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = PromptTemplate.from_template(
    """
    Give a simple recipe for {dish}.

    Requirements:
    - Ingredients
    - Step-by-step preparation
    - Simple English
    """
)

```

```
)

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

parser = StrOutputParser()

chain = prompt | model | parser

dish = input("Enter dish: ")

result = chain.invoke(
    {
        "dish": dish
    }
)

print(result)
```

13. Day 1 Master Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
# Step 1: Create Prompt Template
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} to a {level} learner.
```

```
    Requirements:
```

- Use very simple English
- Give a simple definition
- Give 5 important points
- Give one real-world example
- Give one-line conclusion

```
    """
```

```
)
```

```
# Step 2: Create Chat Model
```

```
model = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

Step 3: Create Output Parser

```
parser = StrOutputParser()
```

Step 4: Create LCEL Chain

```
chain = prompt | model | parser
```

Step 5: Read User Input

```
topic = input("Enter topic: ")
```

```
level = input("Enter learner level: ")
```

Step 6: Execute Chain

```
result = chain.invoke(
```

```
{
```

```
    "topic": topic,
```

```
    "level": level
```

```
}
```

```
)
```

Step 7: Display Result

```
print("\n-----")
```

```
print("FINAL RESPONSE")
```

```
print("-----")
```

```
print(result)
```

14. The Three Most Important Lines (CCE)

14.1 Create Components

prompt = ...

model = ...

parser = ...

14.2 Connect Components

chain = prompt | model | parser

14.3 Execute Chain

result = chain.invoke(input_data)

15. MCQs

MCQ 1

What does LCEL stand for?

- A. LangChain Execution Library
 - B. LangChain Expression Language
 - C. Large Chain Expression Language
 - D. LangChain External Language
-

MCQ 2

What is a Chain?

- A. Only an LLM
 - B. Connected processing components
 - C. Python list
 - D. API key
-

MCQ 3

What does this represent?

`prompt | model`

- A. Delete prompt
 - B. Connect prompt output to model input
 - C. Compare prompt with model
 - D. Python comment
-

MCQ 4

Which order is correct for our basic chain?

- A. Parser → Model → Prompt
 - B. Model → Prompt → Parser
 - C. Prompt → Model → Parser
 - D. Parser → Prompt → Model
-

MCQ 5

What does:

`chain.invoke(input_data)`

do?

- A. Only creates the chain
 - B. Executes the chain for an input
 - C. Installs LangChain
 - D. Creates an API key
-

MCQ 6

If the final component is:

StrOutputParser()

what type of final output do we normally expect?

- A. AIMessage
 - B. String
 - C. PromptTemplate
 - D. Model object
-

MCQ 7

Is the manual approach wrong?

- A. Yes
 - B. No
-

16. Final Summary

1. A Chain connects multiple processing components.
2. Composition means combining smaller components into a larger workflow.
3. LCEL stands for:
LangChain Expression Language
4. LCEL provides a convenient declarative way to compose Runnable components.
5. The pipe operator is: |
6. Main rule:

Output of Left

↓

Input of Right

7. Our basic chain is:

chain = prompt | model | parser

8. Creating a chain is different from executing it.

Create:

chain = prompt | model | parser

Execute:

chain.invoke(...)

9. invoke() processes one input and returns an output.
10. Manual execution is valid.
LCEL provides cleaner pipeline composition.